

第8回: 複数会話の切り替え + 仕上げ

LLMアプリケーション基礎 — 最終回

今日のゴール

ChatGPT風の「複数会話を切り替えて使える」 chat-app を **完成** させる

今日の流れ

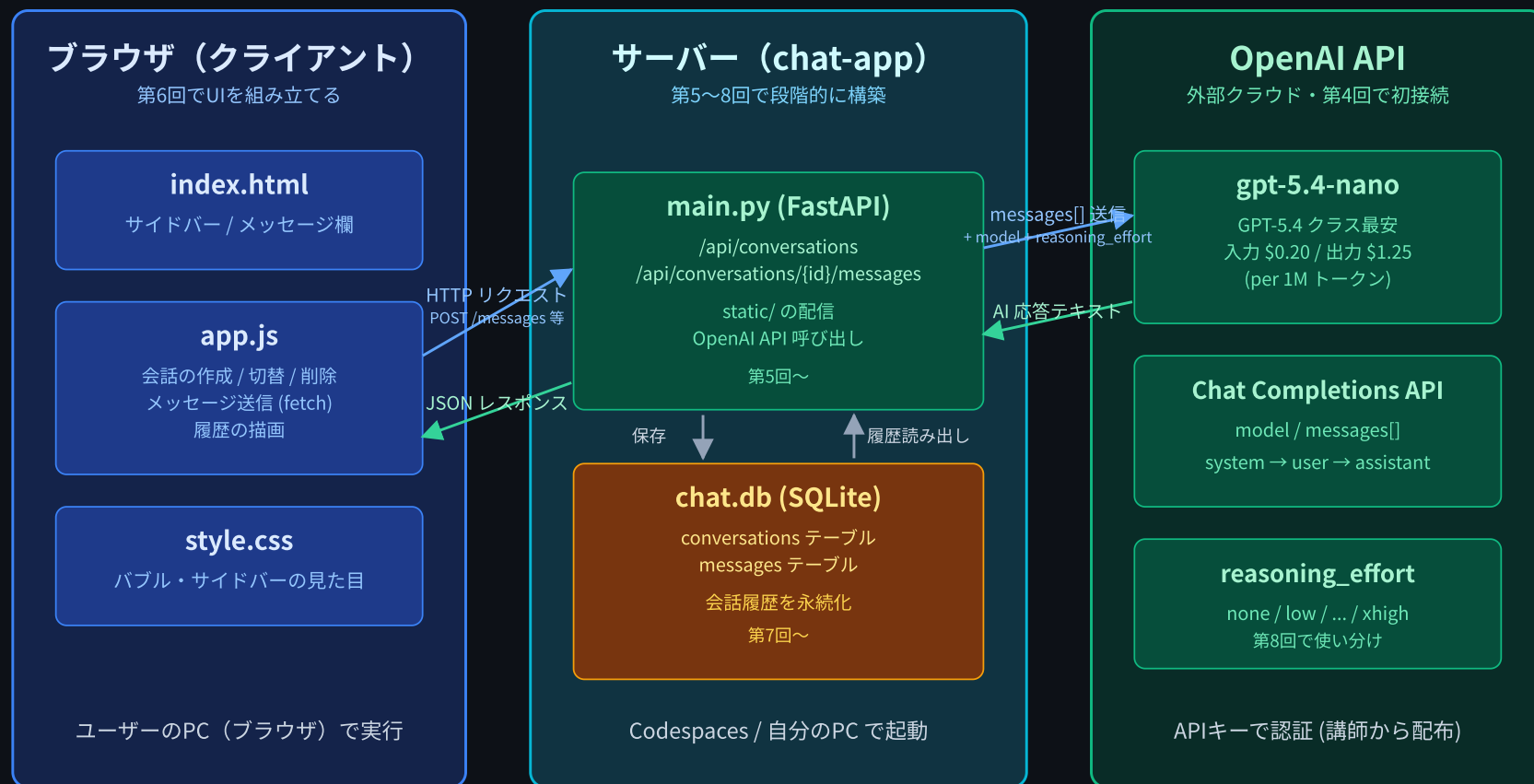
前半

- 前回（第7回）のおさらい
- `conversations` テーブルの追加と外部キー設計
- 会話単位の API 再設計

後半

- サイドバー UI（一覧 / 新規作成 / 切替 / 削除）
- 仕上げ（system プロンプト / Reasoning の使い分け）
- 発展テーマ概観: **Tool use / RAG / エージェント**
- コースの総括と次のステップ

chat-app の全体像



今日は左に **サイドバー UI** を作り、中央に **conversations テーブル** を追加して完成させる

前回 (第7回) のおさらい

- `messages` テーブルを作って 1つの会話を SQLite に永続化 した
- サーバ再起動 / ブラウザリロードしても会話が消えない状態まで来た
- でも... 1つの会話しか持てない

第7回まで: `messages` テーブル 1つだけ

`messages` テーブル

id	role	content	created_at
1	user	こんにちは	2026-05-28 10:00
2	assistant	こんにちは!	2026-05-28 10:00
3	user	Pythonとは?	2026-05-28 10:05



全部ひとつなぎ。話題を分けたい!

→ 第8回で `conversations` テーブルを足して切り分ける

今日やること

メインテーマ:

1. `conversations` テーブル を追加する
2. `messages` に `conversation_id` 外部キー を追加する
3. 会話単位 で API を設計し直す
4. サイダー UI を作る (会話一覧・新規作成・削除)
5. 会話の切り替えと アクティブ表示

最後20分 (軽め):

- システムプロンプトでペルソナを変える / Reasoning の使い分け再訪
- 発展テーマ概観: **Tool use / RAG / エージェント**
- コースの総括と次のステップ

完成形のイメージ

2カラム構成: サイドバー + チャットエリア



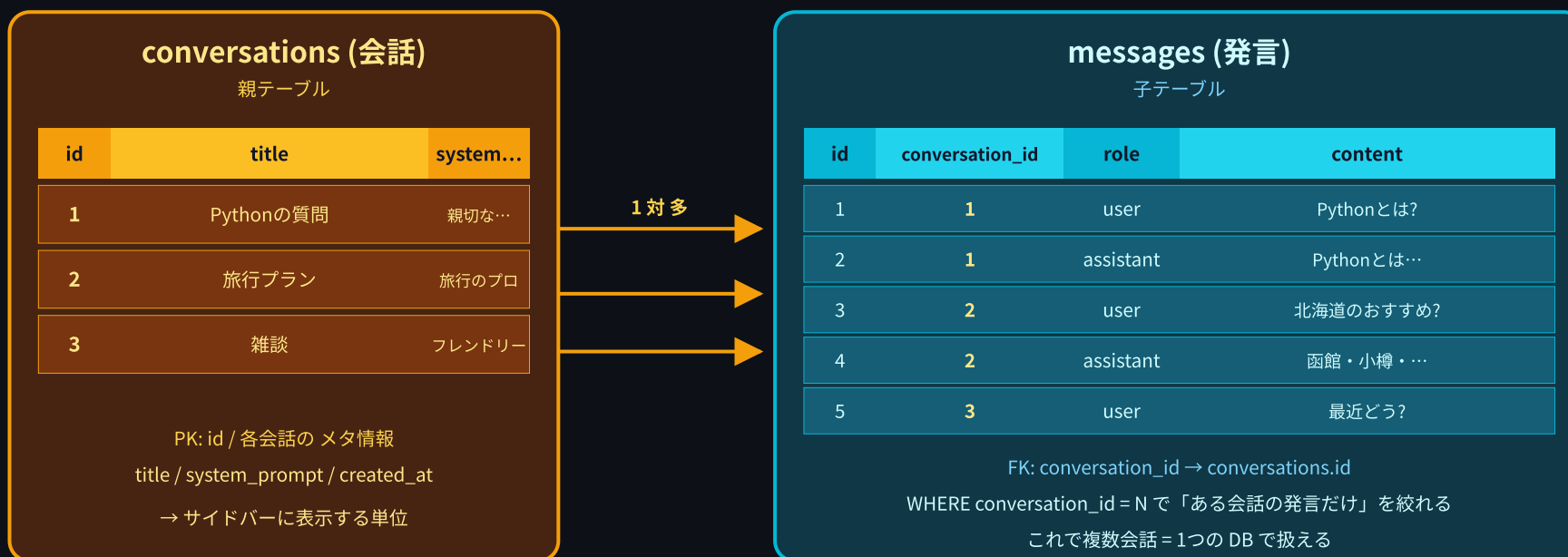
サイドバー
固定幅 260px

チャットエリア
伸縮 (flex: 1)

ChatGPT / Claude の Web 画面と同じレイアウト

データモデルの拡張: 2テーブル構成

2テーブル構成: conversations (親) 1対多 messages (子)



1つの会話 (conversation) は 複数のメッセージ (messages) を持つ

SQL スキーマ

```
CREATE TABLE IF NOT EXISTS conversations (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  title TEXT NOT NULL,  
  system_prompt TEXT NOT NULL,  
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE IF NOT EXISTS messages (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  conversation_id INTEGER NOT NULL,  
  role TEXT NOT NULL,  
  content TEXT NOT NULL,  
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (conversation_id) REFERENCES conversations(id)  
);
```

- `conversations.system_prompt` : その会話固有の system プロンプト (ペルソナの素)
- `messages.conversation_id` : どの会話に属するかを示す 外部キー

FOREIGN KEY って何だっけ

「この列の値は別テーブルの主キーを指している」という宣言

```
FOREIGN KEY (conversation_id) REFERENCES conversations(id)
```

- データの整合性を表現する (どこにも紐づかない宙ぶらりんメッセージを防ぐ)
- SQLite ではデフォルトでは強制チェックされない (`PRAGMA foreign_keys=ON` で有効化)
- 今回は 設計の意図を表すドキュメント として書いておく

```
messages.conversation_id = 999 ← conversations に id=999 が無いとき、  
本来あってはいけない状態
```

API 設計: 「会話」というリソースを足す

第7回までの API (1会話前提):

POST	/api/chat	← メッセージ送信
GET	/api/messages	← 全メッセージ取得

第8回の API (複数会話対応):

GET	/api/conversations	← 会話一覧
POST	/api/conversations	← 新規会話作成
DELETE	/api/conversations/{id}	← 会話削除
GET	/api/conversations/{id}/messages	← その会話のメッセージ
POST	/api/conversations/{id}/messages	← その会話にメッセージ送信

URL パスに `conversation_id` が組み込まれる のがポイント

REST 的なリソース設計

URL がそのまま「何に対する操作か」を表す

```
/api/conversations      ← 会話のコレクション
/api/conversations/5    ← id=5 の会話
/api/conversations/5/messages ← id=5 の会話のメッセージ群
```

メソッド	パス	意味
GET	/api/conversations	一覧取得
POST	/api/conversations	新規作成
DELETE	/api/conversations/{id}	削除
GET	/api/conversations/{id}/messages	その配下を取得
POST	/api/conversations/{id}/messages	その配下に追加

階層 (リソースの入れ子) を URL に反映している

Pydantic モデル (リクエストボディ)

```
class ConversationCreate(BaseModel):  
    """新しい会話を作るときのリクエストボディ"""  
    title: str = Field(default="新しい会話", max_length=100)  
    system_prompt: str = Field(  
        default=DEFAULT_SYSTEM_PROMPT, max_length=2000  
    )  
  
class MessageCreate(BaseModel):  
    """メッセージを送るときのリクエストボディ"""  
    content: str = Field(min_length=1, max_length=4000)
```

- `default=` を付けると **省略可能** になる (フロントは `{}` を投げれば OK)
- `Field(max_length=...)` で長さ制限 → 暴走防止

エンドポイント実装①: 会話一覧

```
@app.get("/api/conversations")
def get_conversations():
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()

    cursor.execute("""
        SELECT id, title, created_at
        FROM conversations
        ORDER BY id DESC
    """)
    rows = cursor.fetchall()

    conn.close()
    return [
        {"id": r["id"], "title": r["title"],
         "created_at": r["created_at"]}
        for r in rows
    ]
```

ORDER BY id DESC で新しい会話が上にくる (ChatGPT と同じ)

エンドポイント実装②: 会話作成

```
@app.post("/api/conversations", status_code=201)
def create_conversation(conversation: ConversationCreate):
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()

    cursor.execute(
        "INSERT INTO conversations (title, system_prompt) "
        "VALUES (?, ?)",
        (conversation.title, conversation.system_prompt),
    )
    conn.commit()
    new_id = cursor.lastrowid

    conn.close()
    return {"id": new_id, "title": conversation.title}
```

- `status_code=201`: 「リソースが作成された」を意味する HTTP ステータス
- `cursor.lastrowid` で新しく入った行の id が取れる

エンドポイント実装③: 会話削除

```
@app.delete("/api/conversations/{conversation_id}")
def delete_conversation(conversation_id: int):
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()

    # 存在チェック
    cursor.execute(
        "SELECT id FROM conversations WHERE id = ?", (conversation_id,)
    )
    if cursor.fetchone() is None:
        conn.close()
        raise HTTPException(404, "Conversation not found")
    # 中のメッセージを先に消す
    cursor.execute(
        "DELETE FROM messages WHERE conversation_id = ?",
        (conversation_id,)
    )
    cursor.execute(
        "DELETE FROM conversations WHERE id = ?", (conversation_id,)
    )
    conn.commit()

    conn.close()
    return {"message": "Conversation deleted", "id": conversation_id}
```


エンドポイント実装④: メッセージ一覧

```
@app.get("/api/conversations/{conversation_id}/messages")
def get_messages(conversation_id: int):
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()

    # 会話が存在するかチェック
    cursor.execute(
        "SELECT id FROM conversations WHERE id = ?", (conversation_id,)
    )
    if cursor.fetchone() is None:
        conn.close()
        raise HTTPException(404, "Conversation not found")

    cursor.execute("""
        SELECT id, role, content, created_at
        FROM messages
        WHERE conversation_id = ?
        ORDER BY id
    """, (conversation_id,))
    rows = cursor.fetchall()

    conn.close()
    return [dict(r) for r in rows]
```

エンドポイント実装⑤: メッセージ送信 (前半)

```
@app.post("/api/conversations/{conversation_id}/messages", status_code=201)
def send_message(conversation_id: int, user_message: MessageCreate):
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()

    # 1. 会話の存在チェック + system_prompt を取り出す
    cursor.execute(
        "SELECT * FROM conversations WHERE id = ?", (conversation_id,)
    )
    conversation = cursor.fetchone()
    if conversation is None:
        conn.close()
        raise HTTPException(404, "Conversation not found")

    # 2. ユーザメッセージを保存
    cursor.execute(
        "INSERT INTO messages (conversation_id, role, content) "
        "VALUES (?, ?, ?)",
        (conversation_id, "user", user_message.content)
    )
    conn.commit()
```

エンドポイント実装⑤: メッセージ送信 (後半)

```
# 3. この会話の過去メッセージ全部を取り出す
cursor.execute("""
    SELECT role, content FROM messages
    WHERE conversation_id = ? ORDER BY id
""", (conversation_id,))
past = cursor.fetchall()

# 4. system + 過去メッセージ全部を API に送る
messages_for_api = [
    {"role": "system", "content": conversation["system_prompt"]},
] + [{"role": r["role"], "content": r["content"]} for r in past]

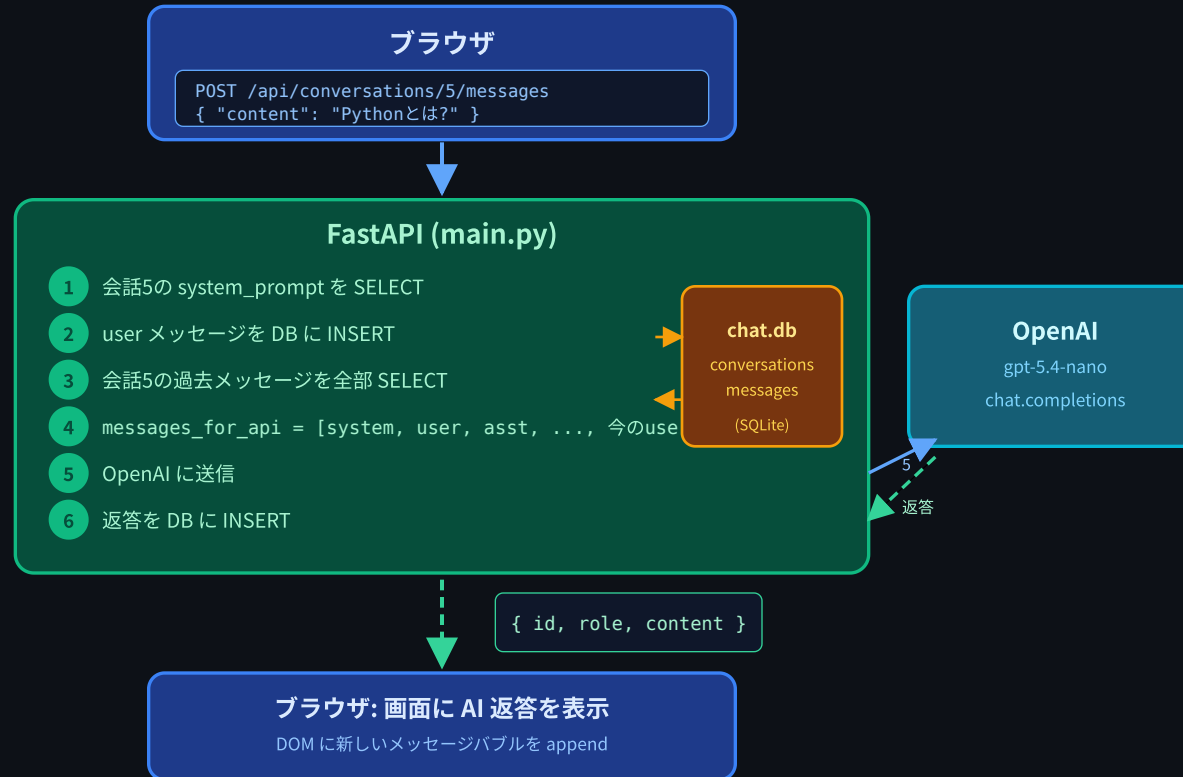
response = client.chat.completions.create(
    model=MODEL_NAME, messages=messages_for_api,
    reasoning_effort=REASONING_EFFORT)

# 5. AI の返答を保存して返す
assistant_content = response.choices[0].message.content
cursor.execute(
    "INSERT INTO messages (conversation_id, role, content) "
    "VALUES (?, ?, ?)",
    (conversation_id, "assistant", assistant_content))
conn.commit()
assistant_id = cursor.lastrowid

conn.close()
return {"id": assistant_id, "role": "assistant",
        "content": assistant_content}
```

1回の送信で起きる流れ (図解)

1回の送信で起きる流れ: 会話単位 API



「毎回まるごと送る」のは第6回と同じ・会話ごとに範囲を絞るだけ

「毎回まるごと送る」のは第6回と同じ。会話ごとに範囲を絞るだけ

サイドバー UI: HTML 構造

```
<div class="app">
  <!-- 左: サイドバー -->
  <aside class="sidebar">
    <button id="new-chat-button">+ 新しい会話</button>
    <ul id="conversation-list"></ul>
  </aside>

  <!-- 右: チャットエリア -->
  <main class="chat-area">
    <div id="message-list"></div>
    <form id="chat-form">
      <textarea id="chat-input"></textarea>
      <button type="submit">送信</button>
    </form>
  </main>
</div>
```

2カラムレイアウト。サイドバーは固定幅、チャットエリアは伸縮

サイダー UI: CSS (抜粋)

```
.app {  
  display: flex; /* 横並び */  
  height: 100vh; /* 画面の高さいっぱい */  
}  
  
.sidebar {  
  width: 260px; /* 固定幅 */  
  background-color: #1f2937;  
  color: white;  
  display: flex;  
  flex-direction: column;  
  padding: 16px;  
}  
  
.chat-area {  
  flex: 1; /* 残りを全部使う */  
  display: flex;  
  flex-direction: column;  
}
```

`display: flex` の基本 (前コースで扱った内容)

フロントの状態管理

```
// 今選択中の会話ID (まだ選んでなければ null)
let currentConversationId = null;

// API 送信中フラグ (連打防止)
let isSending = false;
```

画面に表示される内容 は基本的にこの2つの状態 + サーバから取ったデータで決まる

- `currentConversationId` が `null` → 「会話を選んでください」
- `currentConversationId` が `5` → 会話5のメッセージを描画
- `isSending` が `true` → 送信ボタン無効化

会話一覧の描画 (XSS対策つき)

```
function renderConversations(conversations) {
  const list = document.getElementById("conversation-list");
  list.innerHTML = "";

  conversations.forEach((conv) => {
    const li = document.createElement("li");
    li.className = "conversation-item";
    if (conv.id === currentConversationId) {
      li.classList.add("active"); // 選択中は強調
    }

    const title = document.createElement("span");
    title.textContent = conv.title; // ← textContent で XSS防止
    title.addEventListener("click", () => selectConversation(conv.id));

    li.appendChild(title);
    list.appendChild(li);
  });
}
```

innerHTML ではなく createElement + textContent を使う (XSS 対策)

会話の作成フロー

```
async function createConversation() {  
  const response = await fetch("/api/conversations", {  
    method: "POST",  
    headers: { "Content-Type": "application/json" },  
    body: JSON.stringify({}), // ← 空でOK。サーバ側でデフォルト  
  });  
  const newConv = await response.json();  
  
  // 作った会話を今の会話に切り替える  
  currentConversationId = newConv.id;  
  await loadConversations(); // サイドバー再描画  
  await loadMessages(newConv.id); // 中身は空のはず  
}
```

「作ったら、その会話に切り替える」のがユーザにとって自然

会話の選択 (切り替え) フロー

```
async function selectConversation(conversationId) {  
  currentConversationId = conversationId;  
  // サイドバーの「アクティブ表示」を更新するため再描画  
  await loadConversations();  
  // 選んだ会話のメッセージを読み込む  
  await loadMessages(conversationId);  
}
```

- `currentConversationId` を書き換える
- サイドバーを再描画 (アクティブの位置が動く)
- メッセージ一覧を取り直して表示

会話の削除フロー (確認ダイアログ込み)

```
async function deleteConversation(conversationId) {  
  if (!confirm("この会話を削除しますか?")) {  
    return; // ← ユーザがキャンセルしたら何もしない  
  }  
  
  await fetch(`/api/conversations/${conversationId}`, {  
    method: "DELETE",  
  });  
  
  // 削除したのが今選択中の会話なら、選択を解除する  
  if (currentConversationId === conversationId) {  
    currentConversationId = null;  
    clearMessages();  
  }  
  await loadConversations();  
}
```

`window.confirm` で 誤操作防止。本格的なアプリならカスタムモーダルにする

空状態 (empty state) の表示

「まだ何もない」状態をちゃんとデザインするとアプリの親切度が上がる

- 起動直後 (会話を1つも選んでいない): 「左のメニューから会話を選ぶか、+ 新しい会話 を押してください」
- 新しい会話を作った直後 (メッセージ0件): 「下の入力欄からメッセージを送ってみよう」

```
function clearMessages() {  
  const list = document.getElementById("message-list");  
  list.innerHTML = "";  
  const empty = document.createElement("div");  
  empty.className = "empty-state";  
  empty.textContent = "左のメニューから...";  
  list.appendChild(empty);  
}
```

送信ボタンの連打防止

```
let isSending = false;

async function sendMessage() {
  if (isSending) return; // ← 既に送信中なら何もしない
  isSending = true;
  setSendButtonEnabled(false); // ← ボタンを灰色に
  try {
    // ... API 呼び出し
  } finally {
    isSending = false;
    setSendButtonEnabled(true);
  }
}
```

ネットが遅いときの **二重送信** を防ぐ。地味だが重要

送信フロー全体

1. ユーザの入力を取得
2. 会話が選ばれてなければ → 自動で新規作成
3. ユーザメッセージを画面に追加
4. 「考え中…」をプレースホルダで表示
5. `POST /api/conversations/{id}/messages`
6. レスpons到着
→ 「考え中…」を AI の返答で置き換え
7. エラーが起きたら → エラー表示 + プレースホルダ削除

ユーザ体験の観点では **4 (考え中...)** が大事。送信した瞬間に反応がないと「壊れた?」と感じる

エラーハンドリングのまとめ

場所	何をするか
<code>try/catch</code> 全体	通信エラーをキャッチして <code>showError</code>
<code>response.ok === false</code>	サーバが 4xx/5xx を返した場合
<code>confirm</code>	破壊的操作の前にユーザ確認
<code>isSending</code> フラグ	連打防止
<code>textContent</code>	XSS 防止 (<code>innerHTML</code> を避ける)
<code>Field(max_length=...)</code>	サーバ側で長すぎる入力を拒否

「起きうる失敗を1個1個潰す」のがアプリ開発の地味で大事な仕事

完成形デモ (画面遷移)

1. アプリを開く → サイドバー空 + 「+ 新しい会話」 だけ
2. 「+ 新しい会話」 を押す → 会話が作成され、その会話がアクティブに
3. メッセージを送る → AI が返答
4. もう一度「+ 新しい会話」 → 別の話題で別会話を開始
5. サイドバーの古い会話をクリック → **過去の会話に戻れる**
6. × ボタンで削除 → 確認ダイアログ → 消える

第1回で見せた「完成形デモ」と同じ画面に到達した瞬間

Swagger UI でも確認できる

`http://localhost:8000/docs` を開くと、追加した API が全部見える

- `GET /api/conversations`
- `POST /api/conversations`
- `DELETE /api/conversations/{id}`
- `GET /api/conversations/{id}/messages`
- `POST /api/conversations/{id}/messages`

ブラウザの UI と Swagger UI、両方から同じ API を叩ける ことを確認

==== ここまでがメイン ====

完成形の chat-app ができた

このあとは「じゃあこれをどう発展させていくの？」の話

システムプロンプトでペルソナを変える

conversations テーブルには 既に system_prompt カラムがある

```
class ConversationCreate(BaseModel):  
    title: str = Field(default="新しい会話")  
    system_prompt: str = Field(default=DEFAULT_SYSTEM_PROMPT)
```

会話作成時に違う system_prompt を渡せば、その会話だけ別人格になる

```
curl -X POST http://localhost:8000/api/conversations \  
-H "Content-Type: application/json" \  
-d '{"title": "厳しい先生",  
    "system_prompt": "あなたは厳しい数学教師です。答えを直接教えず、ヒントだけ与えてください。"}'
```

第2回で学んだプロンプトエンジニアリングの知識が活きる場面

ペルソナの例

会話タイトル	system_prompt
雑談	あなたはフレンドリーな話し相手です。
コード相談	あなたはシニアの Python エンジニアです。コードを書くときは型ヒントとdocstringを必ずつけてください。
英語学習	You are an English teacher. Reply in simple English and add a Japanese translation in parentheses.
ニュース要約	あなたはニュース要約アシスタントです。出力は箇条書き3点、各点50字以内になしてください。

UI で system_prompt を編集できるようにする拡張は、次のステップとして楽しいテーマ

Reasoning の使い分け再訪 (第4回の続き)

`gpt-5.4-nano` は **Reasoning 対応** モデル。`reasoning_effort` で考える深さを切り替えられる

```
REASONING_EFFORT = "low"    # ← main.py の1行を変えるだけ
```

値	速度	コスト	用途
<code>none</code>	最速	最安	単純な質問・分類・抽出
<code>low</code>	速い	安い	チャット用途の基本 (今ここ)
<code>medium</code>	中	中	やや込み入った相談
<code>high</code>	遅い	高い	コードレビュー・複雑な推論
<code>xhigh</code>	最遅	最高	難問・論文読解レベル

会話ごとに変える という発想が次の自然な拡張

会話ごとに reasoning_effort を変える

これは現状の実装ではなく、次の一步としての 応用案 です(現在の chat-app の DB スキーマには reasoning_effort カラムは存在しません)

```
# 案: conversations テーブルに reasoning_effort カラムを足す
CREATE TABLE conversations (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  system_prompt TEXT NOT NULL,
  reasoning_effort TEXT NOT NULL DEFAULT 'low', -- ← 追加
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

```
# 送信時
response = client.chat.completions.create(
  model=MODEL_NAME,
  messages=messages_for_api,
  reasoning_effort=conversation["reasoning_effort"], # ← 会話ごと
)
```

雑談用は none / low、コード相談は high / xhigh のように使い分ける

「コードに1行追加するだけで挙動が変わる」 デモ

```
# パターンA: 雑談用の会話  
reasoning_effort="none" # 即レス。トークン消費も最小  
  
# パターンB: コードレビュー用の会話  
reasoning_effort="high" # 数秒～十数秒待つが、深く考えた回答が返る
```

同じ質問

「このコードの計算量を改善する方法は？」

を A/B で投げると **応答の質・レイテンシ・トークン数** に明確な差が出る

LLM アプリでは「どこに頭を使わせるか」がコスト設計 の半分以上を占める

===== ここから発展テーマ =====

ここまでで作った chat-app は「ただ会話するだけ」のアプリ
世の中で動いている AI アプリは、これに $+\alpha$ を載せている

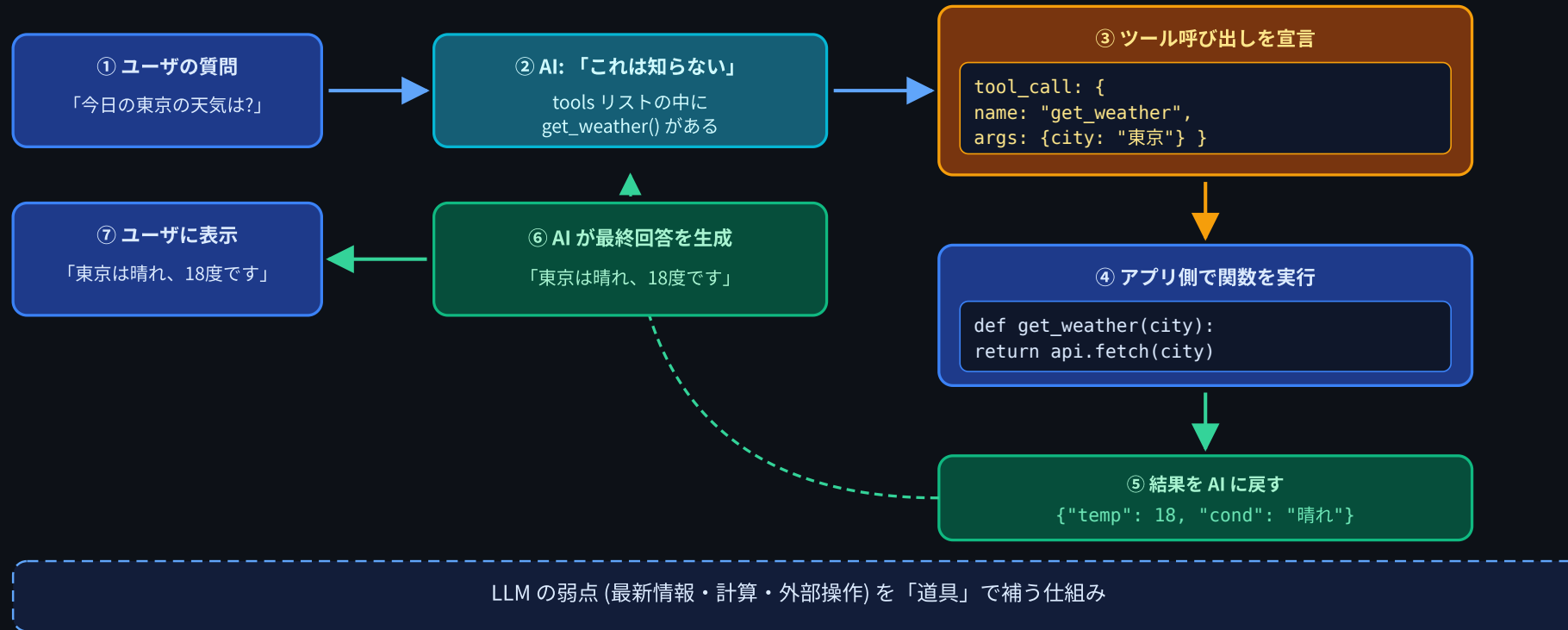
主な拡張パターン:

1. **Tool use / Function calling** — AI に「使える道具」を渡す
2. **RAG** — 自分のドキュメントを参照させる
3. **エージェント** — AI に自律的に動いてもらう

(以降は概念紹介。実装はこのコースの範囲外)

発展① Tool use / Function calling

Tool use / Function calling — AI に道具を渡す



LLM の弱点 (最新情報を知らない・計算が苦手) を 道具で補う

Function calling のイメージコード

```
tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "指定された都市の現在の天気を返す",
        "parameters": {
            "type": "object",
            "properties": {"city": {"type": "string"}},
            "required": ["city"],
        },
    },
}]

response = client.chat.completions.create(
    model=MODEL_NAME,
    messages=messages,
    tools=tools,      # ← ここで「使える道具リスト」を渡す
)
# AI が呼びたい関数と引数を申告 → アプリ側が実行 → 結果を AI に戻す
```

ChatGPT が Web 検索・コード実行・画像生成を呼んでいるのは、これの拡張

発展② RAG (Retrieval Augmented Generation)

「自分のドキュメントを参照させる」 仕組み

[ユーザ] 「弊社の有給休暇の取り方は？」

↓

[ふつうの LLM] → 知らないので一般論を答える / ハルシネーション

↓

[RAG]

- ① 質問に近い社内ドキュメントを検索（ベクトル検索）
- ② ヒットしたドキュメントをプロンプトに添付
- ③ 「以下の資料を参考に回答してください： <検索結果>」 として LLM に渡す

↓

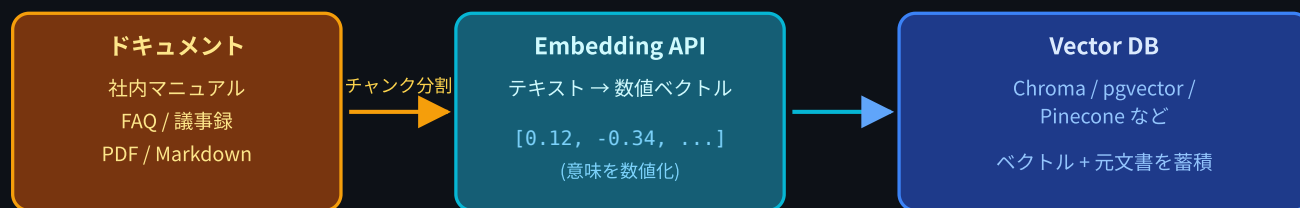
[LLM] → 社内資料に基づいた正確な回答

第2回の「必要な前提・文脈をプロンプトに渡せば精度が上がる」を自動化したもの

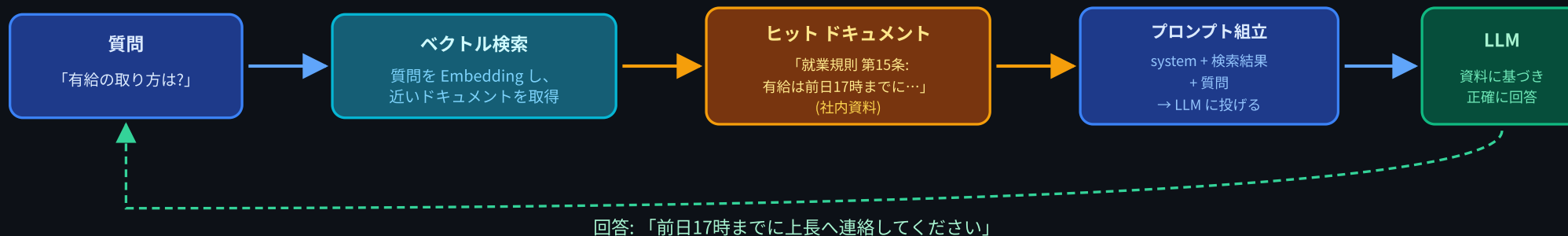
RAG のアーキテクチャ

RAG (Retrieval Augmented Generation) — 自分のドキュメントを参照させる

事前準備: ドキュメントをベクトル化して保存 (オフライン)



実行時: 質問が来たら検索 → プロンプトに添付して LLM へ



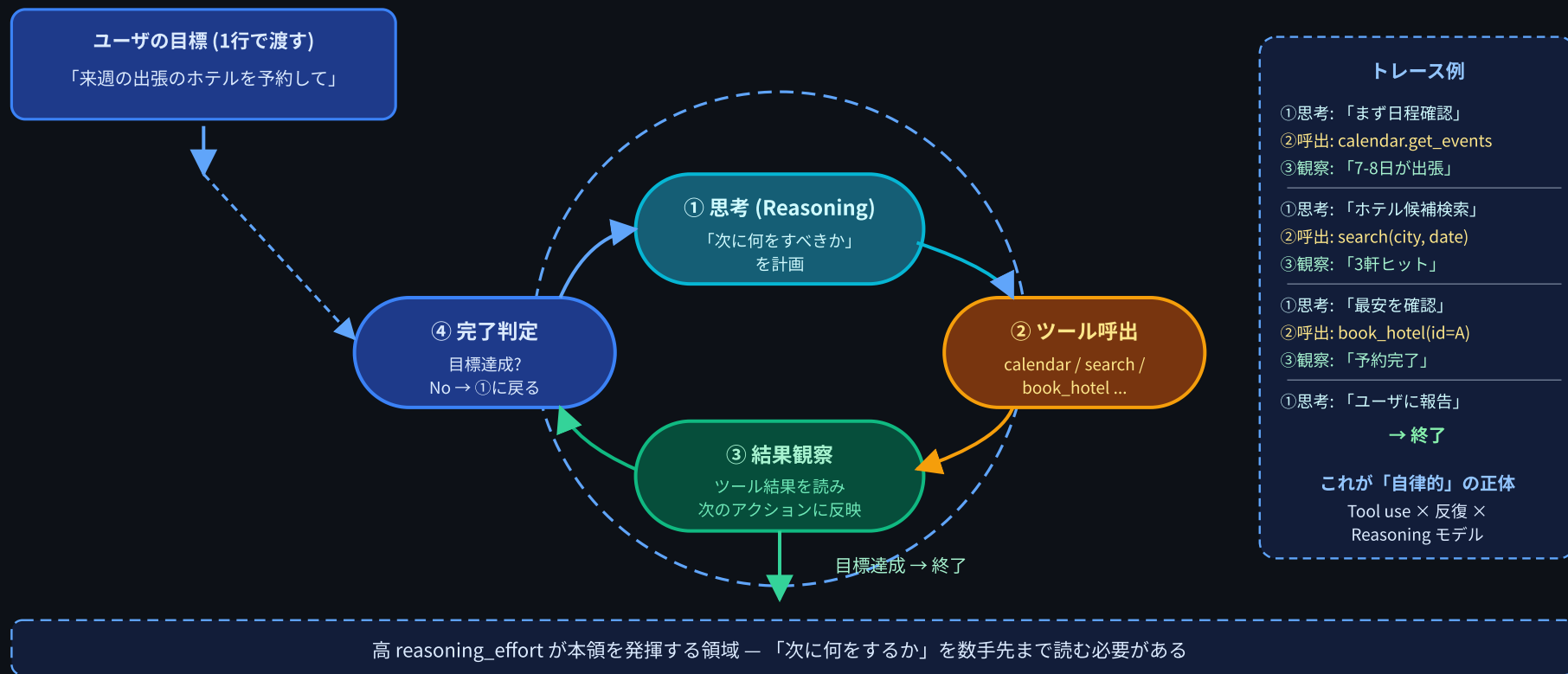
回答: 「前日17時までに上長へ連絡してください」

第2回の「プロンプトに必要な文脈を渡せば精度が上がる」を自動化 / 社内 Q&A bot ・ FAQ ・ 論文検索 等

- ベクトル化 = テキストを意味的な数値ベクトルに変換すること (Embedding API)
- 用途: 社内 FAQ、製品マニュアル、議事録検索、論文 Q&A 等

発展③ エージェント

エージェント — Tool use を「ループ」で回し、自律的に動く



Tool use を繰り返し回すループ + 計画立て = エージェント

エージェントと Reasoning モデル

エージェントは「今何をすべきか」を自分で考える必要がある

→ ここで **Reasoning モデル** が土台になる

- 「次にどのツールを呼ぶか」を決めるには **計画** が要る
- 単純な応答よりも **数手先を読む** 能力が重要
- だから `reasoning_effort="high"` 系のモードが本領を発揮する

雑談チャット → `reasoning_effort="none"~"low"`
RAG（検索+応答）→ `reasoning_effort="low"~"medium"`
エージェント → `reasoning_effort="high"~"xhigh"`

「頭を使うほどコストとレイテンシが上がる」のは人間と同じ

発展テーマの組み合わせ

実際の AI アプリは複数を組み合わせている

例	組み合わせ
ChatGPT (Web版)	Tool use (検索・画像生成・コード実行)
社内 Q&A bot	RAG + chat-app
AI コーディング支援	エージェント + Tool use (ファイル読み書き・テスト実行)
カスタマーサポート bot	RAG + Tool use (チケット作成)

今日完成させた chat-app は、これらすべての土台になる

このコースで身についたこと

1. LLM の正体 — 次のトークンを予測するモデル / 得意・不得意
 2. プロンプトエンジニアリング — system / user / few-shot / 構造化出力
 3. Reasoning モデル — 考える深さを切り替えるという発想
 4. OpenAI API の使い方 — Python から呼ぶ、トークンとコストの感覚
 5. FastAPI でラップ — API キー保護のための バックエンド経由 の理由
 6. マルチターン会話 — 履歴を `messages` 配列に積む
 7. SQLite で永続化 — リロードしても残るアプリ
 8. 複数会話の管理 — リソース設計、サイドバー UI、状態管理
- ChatGPT 風アプリを自分で作れる人

このコース修了後にできること

- 自分のアイデアの AI アプリを試作できる
 - 社内ツール、個人のアシスタント、英語学習 bot...
- 既存の AI アプリの中身を想像できる
 - 「あ、これ system プロンプトで人格決めてるな」「ここ Tool use 使ってそう」
- AI 関連の技術記事を読み解ける
 - RAG / エージェント / Function calling といった単語の意味が分かる
- コストとレイテンシを意識した設計ができる
 - トークン量、Reasoning レベル、コンテキストウィンドウ

次に学ぶといいこと (技術トピック)

興味の方向	学ぶといいもの
応答品質を上げたい	プロンプトエンジニアリング (深掘り) / 評価 (Eval)
自分のデータを使いたい	RAG / Embedding / Vector DB (Chroma, pgvector)
自動化したい	Tool use / Function calling / エージェント
より速く / 安く	ストリーミング応答 / Prompt Caching / モデル選定
本番運用	レート制限 / 監視 / ログ / コスト管理 / 認証
他のモデル	Claude (Anthropic) / Gemini (Google) / OSS (Llama 等)

次に学ぶといいこと (実装トピック)

このコースで省略したもの:

- ストリーミング応答 (`stream=True`) — タイプライター風に少しずつ表示
- 会話タイトルの自動生成 — 最初のメッセージから AI にタイトルを付けさせる
- 会話のエクスポート / 検索
- ユーザ認証 — マルチユーザ対応
- 本番デプロイ — Railway / Render / Cloudflare Workers / Fly.io 等
- フロントエンドフレームワーク — React / Vue で SPA 化

「作って動かす」のが一番の学習法。何か1つ作ってみよう

自分のアプリを作るときのチェックリスト

- ☐ どんなユーザの、どんな困りごとを解決するか を1行で書ける
- ☐ **system** プロンプト の方針が決まっている
- ☐ **API キー** はサーバ側でしか持たない
- ☐ どんな入力を受け付ける か (バリデーション)
- ☐ どう永続化するか (SQLite で十分? Postgres が必要?)
- ☐ **コスト試算** — 1ユーザ・1日あたりのトークン消費は?
- ☐ **エラー時の挙動** — API が落ちた / 上限超え時はどうする?
- ☐ **Reasoning** はどのレベルが妥当?

本日のまとめ

学んだこと

1. `conversations` + `messages` の2テーブル構成で複数会話を扱える
2. URLを `/api/conversations/{id}/messages` の **リソース指向** に再設計
3. **サイドバーUI** で会話の一覧・新規作成・切替・削除
4. ChatGPT風の chat-app が **完成**
5. 次のステップ — **Tool use / RAG / エージェント** の地図を持った

最後に

このコースで作った chat-app は 小さい けど、

- LLM を呼ぶ
- 会話を管理する
- データを永続化する
- 複数のリソースを REST で扱う
- 失敗を扱う

という、AI アプリのコア要素 を全部体験した。

ここから先は 自分が作りたいものを作るフェーズ です。

好きなテーマで、自分の chat-app から派生させてみてください。

提出物

完成した chat-app をフォームから提出してください:

1. session08/exercise/ の chat-app (完成版) の GitHub のURL
 - 例: `https://github.com/ユーザー名/リポジトリ名/tree/main/session08/exercise`

お疲れ様でした！

8回のコース、最後までよく走りきりました。

LLMアプリケーション基礎 — 完

質問・感想・作ったものの共有、いつでも歓迎です。